

移行可能なアプリケーションのアーキテクチャ選定について考えてみる。

Spring Boot が必要とされること自体が、Spring の肥大化を物語っている。

Java における Web アプリケーション技術の変遷

RPC

業務システムではシステム間で連携することが多々あるが、JSON や YAML のようなシンプルなデータフォーマットは存在せず、現在のような REST API はまだ主流ではなかった。

システム間でやりとりするための規格として CORBA があったが、CORBA の仕様は複雑で巨大であったため、CORBA のライブラリを提供しているベンダとサポート契約などを結んで使用していた。

RMI

Java 限定ではあったが、システム間のやりとりのための仕様として RMI が新たに定義され、CORBA よりは楽に RPC ができるようになったが、それでも HTTP で REST API を呼び出すことに比べれば準備も使用方法も煩雑に過ぎた。

J2EE

そのような RMI の使い方に一定のルールを設け、EJB を始め様々な指針を提示したものとして J2EE が登場した。

J2EE は、Java 2 Platform, Enterprise Edition の略で、サンマイクロシステムズ社が策定した Java による企業システム構築のための仕様である。スタンドアロンのアプリケーションを開発するときに使う J2SE (Java2 Platform, Standard Edition) を、エンタープライズシステムに対応するように拡張した仕様となる。

J2EE にはビジネス・ロジックのコンポーネント化を目的とした EJB、Servlet や JSP といった CGI の代替となるように策定された Web アプリケーション構築用の API などが含まれており、エンタープライズシステム設計の指針が提示されたことで実装者がそれぞれに試行錯誤して車輪を再開発するような無駄は減ったが、その代償として仕様はバージョンを重ねるたびに複雑化していった。

Spring の登場

実現したい機能に対して、仕様が悪夢のように複雑な J2EE のアンチテーゼとして登場したのが Spring である。Spring 自体は DI (Dependency Injection)、直訳すると「依存性の注入」というパターンを実現するためのフレームワークであり、このようなフレームワークは DI コンテナと呼ばれる。

DI とは、オブジェクトが依存する他のオブジェクトを受け取るデザインパターンであり、オブジェクトの作成と利用について関心の分離を行い、疎結合なプログラムを実現することを目的としている。

歴史は繰り返す

アプリケーションの構成はどのように明示されるべきか

XML 地獄

J2EE の複雑さに対する解決策として登場した Spring だが、デメリットが無いわけではなく、当然ながら DI コンテナ特有の問題があった。

DI コンテナがインスタンスを生成・管理するために、DI の対象となるオブジェクトとそれらの依存関係は全て DI コンテナに登録されていなくてはならない。登録するオブジェクトが多ければ多いほど設定する項目数は膨れ上がり、DI コンテナの起動時に解決しなければならない依存関係も複雑になるため、起動が遅くなった。

当時は XML が設定ファイルのフォーマットとして主流だったため、Spring のような DI コンテナに登録するオブジェクトのライフサイクルや依存関係を定義した膨大な XML ファイルを大量に作成しなければならなくなり、そういった様相は「XML 地獄」と呼ばれていた。